CI/CD & Automatisation DevOps

Jenkins · GitLab · Docker · SonarQube · Maven

Formation complète

Sommaire

1 Le concept d'intégration continue

2 GitLab — La gestion du code source

3 Jenkins — L'automatisation de l'intégration

4 La gestion des binaires et des dépendances

5 La conteneurisation avec Docker

6 La sécurité et la qualité du code

7 Simplifier l'intégration avec les outils disponibles

01
Le concept d'intégration continue

Comprendre le DevOps et la CI/CD

Qu'est-ce que le DevOps ?

Définition

Le DevOps est une culture visant à rapprocher les équipes de Développement (Dev) et d'Exploitation (Ops) pour livrer des logiciels plus rapidement et de façon plus fiable.

Les 3 piliers

Culture de collaboration, Automatisation des processus, Mesure et amélioration continue.

Le cycle de vie

Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → (retour à Plan)

Bénéfices mesurés

Les organisations DevOps déploient 46x plus fréquemment et ont un taux d'échec 5x plus faible que les organisations traditionnelles

Les enjeux de la livraison logicielle

Modèle traditionnel ❌

- Cycles de développement longs (6-12 mois)
- Intégration tardive → conflits massifs
- Bugs découverts en production
- Déploiements manuels risqués
- Silos entre Dev et Ops

Modèle CI/CD ✅

- Livraisons fréquentes (quotidiennes ou plus)
- Intégration continue → conflits mineurs
- Bugs détectés dès le commit
- Déploiements automatisés et reproductibles
- Collaboration Dev/Ops permanente

L'Intégration Continue (CI)

Définition

La CI (Continuous Integration) est la pratique consistant à intégrer fréquemment le code dans un dépôt partagé et à déclencher automatiquement une suite de vérifications à chaque commit.

Principe fondateur

"Intégrez tôt, intégrez souvent" — Martin Fowler (2006). Chaque développeur intègre son travail au moins une fois par jour.

Ce que la CI automatise

Compilation du code source, exécution des tests unitaires et d'intégration, analyse statique de la qualité du code.

Le résultat

Un feedback rapide (< 10 minutes) sur la santé du code, permettant de corriger les problèmes au plus tôt.

La Livraison et le Déploiement Continus (CD)

Continuous Delivery

- Le code est toujours dans un état déployable
- La mise en production est une décision humaine
- Chaque build passant les tests peut être livré
- Réduit le risque de chaque déploiement

Continuous Deployment

- Chaque commit validé est automatiquement déployé en production
- Aucune intervention humaine requise
- Nécessite une confiance totale dans les tests automatisés
- Pratiqué par Netflix, Amazon, Google

Les briques d'un pipeline CI/CD

Étape	Rôle	Outil(s) typique(s)
Source Control	Versionner le code source	GitLab, GitHub, Bitbucket
Build	Compiler le code source	Maven, Gradle, npm
Test	Vérifier la correction du code	JUnit, Mockito, Selenium
Analyse Qualité	Détecter bugs et vulnérabilités	SonarQube, Checkstyle
Packaging	Créer l'artefact livrable	Maven (JAR/WAR), Docker
Déploiement	Livrer en environnement cible	Jenkins, Ansible, Kubernetes

Application dans le contexte SNCF

Enjeux SNCF

La SNCF développe des applications critiques (gestion des horaires, billetterie, supervision des trains). La fiabilité et la rapidité de livraison sont essentielles.

Réduction des risques

La CI/CD permet de détecter les régressions avant qu'elles n'atteignent la production, réduisant les incidents critiques.

Time-to-market

Passer de cycles de 6 mois à des livraisons hebdomadaires ou quotidiennes pour répondre plus vite aux besoins métier.

Standardisation

Uniformiser les pratiques de développement entre les équipes via des pipelines partagés et des standards de qualité communs.

02
GitLab

La gestion du code source

Qu'est-ce qu'un SCM ?

Définition

Un SCM (Source Code Management) est un système permettant de gérer les modifications apportées au code source au fil du temps.

Pourquoi versionner ?

Traçabilité complète des modifications, possibilité de revenir à une version antérieure, collaboration simultanée entre développeurs.

Historique

CVS (1990) → Subversion/SVN (2000) → Git (2005, Linus Torvalds) → Hébergeurs Git (GitHub 2008, GitLab 2011).

Git vs SVN

Git est distribué (chaque développeur a une copie complète de l'historique), SVN est centralisé (un seul dépôt de référence).

Introduction à Git

Repository (dépôt)

Répertoire contenant le code source et tout l'historique des modifications. Peut être local (.git/) ou distant (GitLab, GitHub).

Commit

Instantané (snapshot) de l'état du projet à un moment donné. Chaque commit a un identifiant unique (hash SHA-1) et un message descriptif.

Branch (branche)

Ligne de développement indépendante. Permet de travailler sur une fonctionnalité sans impacter le code principal.

Merge (fusion)

Intégration des modifications d'une branche dans une autre. Peut générer des conflits si les mêmes lignes ont été modifiées.

Les commandes Git essentielles

Commande	Description
git init	Initialiser un nouveau dépôt Git local
git clone <url>	Cloner un dépôt distant en local
git add <fichier>	Ajouter des fichiers à la zone de staging
git commit -m 'msg'	Enregistrer les changements avec un message
git push origin main	Envoyer les commits vers le dépôt distant
git pull	Récupérer et fusionner les changements distants
git branch <nom>	Créer une nouvelle branche
git checkout <branche>	Basculer sur une branche existante
git merge <branche>	Fusionner une branche dans la branche courante
git status	Afficher l'état du dépôt (fichiers modifiés, staged...)

GitLab : bien plus qu'un hébergeur Git

Hébergement de code

Interface web pour parcourir le code, l'historique, les branches et les tags. Comparable à GitHub mais auto-hébergeable.

Gestion de projet

Issues (tickets), Milestones, Labels, Boards Kanban — tout pour gérer le cycle de vie d'un projet logiciel.

CI/CD intégrée

Pipelines CI/CD natifs via le fichier .gitlab-ci.yml, sans avoir besoin d'un outil externe comme Jenkins.

Revue de code

Merge Requests (MR) avec commentaires inline, approbations obligatoires, détection de conflits et intégration des résultats CI.

Les workflows Git : GitFlow

main (ou master)	Branche de production. Contient uniquement du code stable et déployé. Protégée contre les push directs.
develop	Branche d'intégration. Reçoit les nouvelles fonctionnalités terminées. Base pour les releases.
feature/*	Créée depuis develop pour développer une fonctionnalité. Fusionnée dans develop via une Merge Request.
release/*	Préparation d'une nouvelle version (corrections de bugs mineurs, mise à jour des numéros de version). Fusionnée dans main ET develop.
hotfix/*	Correction urgente en production. Créée depuis main, fusionnée dans main ET develop.

Les Merge Requests et la revue de code

Qu'est-ce qu'une MR ?

Une Merge Request (MR) est une demande formelle de fusionner une branche dans une autre. Elle centralise la discussion autour des changements proposés.

Processus

1. Développeur crée une branche feature. 2. Ouvre une MR. 3. Les reviewers commentent et approuvent. 4. Les pipelines CI s'exécutent. 5. Fusion si tout est vert.

Bonnes pratiques

MR petites et focalisées (< 400 lignes), titre descriptif, description des changements, lien vers l'issue correspondante.

Protections de branche

Dans GitLab, on peut exiger N approbations, que les pipelines CI passent, et interdire les push directs sur main.

GitLab CI/CD : le fichier .gitlab-ci.yml

Le fichier .gitlab-ci.yml définit le pipeline CI/CD de GitLab. Il est versionné avec le code source.

```yaml
stages:
  - build
  - test
  - deploy

build-job:
  stage: build
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn clean compile -B

test-job:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn test -B
  artifacts:
    reports:
      junit: target/surefire-reports/*.xml

deploy-job:
  stage: deploy
  script:
    - echo "Déploiement en cours..."
  only:
    - main
```

03
Jenkins

L'automatisation de l'intégration continue

Qu'est-ce que Jenkins ?

Définition

Jenkins est un serveur d'automatisation open-source écrit en Java. Il permet d'automatiser toutes les étapes du cycle de vie logiciel : build, test, déploiement.

Historique

Créé en 2004 par Kohsuke Kawaguchi chez Sun Microsystems sous le nom Hudson. Renommé Jenkins en 2011 après un fork suite au rachat de Sun par Oracle.

Écosystème

Plus de 1 800 plugins disponibles. Intégrations avec Git, Maven, Docker, SonarQube, Kubernetes, AWS, Azure, Slack...

Adoption

Jenkins est utilisé par des milliers d'organisations dans le monde, dont de nombreuses grandes entreprises françaises.

Architecture Jenkins : Controller et Agents

Controller (Master)

Le cerveau de Jenkins. Il orchestre les builds, stocke la configuration, gère les plugins et présente l'interface web. Ne doit PAS exécuter les builds directement en production.

Agent (Slave/Node)

Machine (physique, VM ou conteneur) qui exécute les builds délégués par le Controller. Peut être permanent ou éphémère (Docker, Kubernetes).

Communication

Le Controller communique avec les agents via SSH ou via le protocole JNLP (Java Network Launch Protocol) sur le port 50000.

Avantages

Scalabilité horizontale (ajouter des agents), isolation des builds (différents OS, JDK, outils), parallélisation des pipelines.

Les concepts fondamentaux de Jenkins

Concept	Définition
Job / Project	Unité de travail configurable dans Jenkins (compilation, test, déploiement...)
Build	Une exécution d'un Job. Chaque build a un numéro, un statut (SUCCESS, FAILURE, UNSTABLE) et des logs.
Workspace	Répertoire de travail sur l'agent où le code source est cloné et les builds s'exécutent.
Artifact	Fichier produit par un build (JAR, WAR, image Docker) et archivé par Jenkins pour utilisation ultérieure.
Plugin	Extension qui ajoute des fonctionnalités à Jenkins (intégration Git, Docker, SonarQube, notifications...).
Credential	Secret stocké de façon sécurisée dans Jenkins (mot de passe, token, clé SSH) utilisable dans les pipelines.

L'interface de Jenkins

Tableau de bord

Vue d'ensemble de tous les jobs avec leur statut (boule colorée : bleu=succès, rouge=échec, jaune=instable, gris=jamais exécuté).

Vue d'un Job

Historique des builds, lien vers les logs, graphique de tendance (test results, build duration), configuration.

Blue Ocean

Interface graphique moderne pour Jenkins. Visualisation des pipelines sous forme de graphe, éditeur visuel de Jenkinsfile.

Manage Jenkins

Configuration globale : outils (JDK, Maven, Git), plugins, credentials, sécurité, agents, système.

Les types de Jobs Jenkins

Type	Description	Cas d'usage
Freestyle Project	Configuration via l'interface graphique. Simple mais limité.	Jobs simples, scripts shell, tâches ponctuelles.
Pipeline	Décrit le build dans un fichier Jenkinsfile (code Groovy). Versionnable.	Pipelines CI/CD complexes, multi-étapes.
Multibranch Pipeline	Découvre automatiquement les branches d'un dépôt Git et crée un job par branche.	Projets GitFlow avec plusieurs branches actives.
Organization Folder	Découvre automatiquement tous les dépôts d'une organisation GitLab/GitHub.	Gestion de nombreux dépôts dans une organisation.

Le Pipeline as Code

Définition

Le Pipeline as Code consiste à décrire le processus de build dans un fichier texte (Jenkinsfile) versionné avec le code source dans le dépôt Git.

Avantages

Versionnable (historique des changements de pipeline), revue de code possible, reproductible, partageable entre projets.

Deux syntaxes

Scripted Pipeline (Groovy pur, flexible mais complexe) et Declarative Pipeline (structure imposée, plus lisible, recommandée).

Emplacement

Le Jenkinsfile est placé à la racine du dépôt Git. Jenkins le détecte automatiquement lors de la configuration du job.

Syntaxe déclarative du Jenkinsfile

Structure d'un Jenkinsfile déclaratif — la syntaxe recommandée pour les nouveaux projets.

```
pipeline {
    agent { docker { image 'maven:3.9-eclipse-temurin-17' } }

    environment {
        APP_NAME = 'mon-application'
    }

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Compilation') {
            steps { sh 'mvn clean compile -B' }
        }
        stage('Tests') {
            steps { sh 'mvn test -B' }
            post { always { junit 'target/surefire-reports/*.xml' } }
        }
    }

    post {
        success { echo "Build réussi !" }
        failure { echo "Build échoué !" }
    }
}
```

Les instructions Groovy essentielles

Instruction	Description
sh 'commande'	Exécuter une commande shell (Linux/Mac)
bat 'commande'	Exécuter une commande batch (Windows)
echo 'message'	Afficher un message dans les logs
git url: '...', branch: '...'	Cloner un dépôt Git
archiveArtifacts 'target/*.jar'	Archiver des fichiers produits par le build
junit 'target/surefire-reports/*.xml'	Publier les résultats des tests JUnit
withCredentials([...])	Injecter des secrets dans le contexte du build
parallel { stage(...) ... }	Exécuter des stages en parallèle
input message: '...'	Pause manuelle avec validation humaine
cleanWs()	Nettoyer le workspace après le build

Les plugins Jenkins incontournables

Plugin	Rôle
Git Plugin	Intégration avec les dépôts Git (clone, checkout, webhooks)
Pipeline	Support des Jenkinsfiles déclaratifs et scriptés
Docker Pipeline	Utiliser Docker comme agent et construire des images
SonarQube Scanner	Lancer des analyses SonarQube depuis un pipeline
Maven Integration	Support natif de Maven dans les jobs Freestyle
Blue Ocean	Interface graphique moderne pour visualiser les pipelines
Credentials Binding	Injecter des credentials dans les variables d'environnement
Email Extension	Envoyer des notifications email personnalisées
Workspace Cleanup	Nettoyer automatiquement les workspaces

Déclenchement automatique des builds

Webhook (recommandé)

GitLab/GitHub notifie Jenkins à chaque push via une requête HTTP. Déclenchement quasi-instantané. Nécessite que Jenkins soit accessible depuis GitLab.

Polling SCM

Jenkins interroge régulièrement le dépôt Git pour détecter les changements. Simple à configurer mais moins efficace. Syntaxe cron : H/5 * * * *

Planification (cron)

Déclenchement à une heure fixe (ex: build nocturne). Syntaxe : 0 2 * * * (tous les jours à 2h du matin).

Déclenchement par un autre job

Un job peut déclencher un autre job via le plugin 'Build other projects' ou l'instruction build() dans un Jenkinsfile.

TP 1

TP 1 — Installation et découverte de Jenkins

Déployer Jenkins via Docker, configurer le compte administrateur, installer les plugins essentiels et explorer l'interface.

💡 **Indice :** *Utilisez l'image jenkins/jenkins:lts-jdk17 et montez le socket Docker pour les TPs suivants.*

À vous de jouer !

04

La gestion des binaires et des dépendances

Maven et les gestionnaires d'artefacts

Qu'est-ce qu'une dépendance logicielle ?

Définition

Une dépendance est une bibliothèque externe (JAR, module npm...) dont votre projet a besoin pour fonctionner. Ex: Spring Boot, JUnit, Log4j.

Problèmes sans gestionnaire

Téléchargement manuel des JARs, conflits de versions (dependency hell), builds non reproductibles, taille du dépôt Git gonflée.

Solution

Un gestionnaire de dépendances (Maven, Gradle, npm, pip) automatise le téléchargement, la résolution des conflits et la reproductibilité des builds.

Versionnement sémantique

Les dépendances suivent le format MAJOR.MINOR.PATCH (ex: 3.9.2). MAJOR = rupture de compatibilité, MINOR = nouvelles fonctionnalités, PATCH = corrections.

Maven : le gestionnaire de build Java

Définition

Apache Maven est un outil de build et de gestion de projet Java. Il standardise la structure des projets, gère les dépendances et automatise le cycle de build.

Convention over Configuration

Maven impose une structure de projet standard (src/main/java, src/test/java, target/) ce qui rend tous les projets Maven immédiatement compréhensibles.

Le fichier pom.xml

Project Object Model — fichier XML décrivant le projet : identité (groupId, artifactId, version), dépendances, plugins de build, configuration.

Dépôt central

Maven Central Repository (https://repo.maven.apache.org) héberge des millions de bibliothèques Java téléchargeables automatiquement.

Le fichier pom.xml décortiqué

Exemple d'un pom.xml pour un projet Java Spring Boot avec tests JUnit et analyse SonarQube.

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <!-- Identité du projet (GAV) -->
  <groupId>com.sncf.demo</groupId>
  <artifactId>demo-java-app</artifactId>
  <version>1.0.0</version>

  <properties>
    <java.version>17</java.version>
    <sonar.projectKey>demo-java-app</sonar.projectKey>
  </properties>

  <dependencies>
    <!-- Dépendance de test -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Le cycle de vie Maven

Phase	Description	Commande
validate	Valide la structure du projet	mvn validate
compile	Compile le code source Java	mvn compile
test	Exécute les tests unitaires	mvn test
package	Crée le JAR/WAR livrable	mvn package
verify	Exécute les tests d'intégration	mvn verify
install	Installe dans le dépôt local (~/.m2)	mvn install
deploy	Publie sur un dépôt distant (Nexus)	mvn deploy

Le dépôt central Maven (Maven Central)

Dépôt local

~/.m2/repository — Cache local sur la machine du développeur ou de l'agent Jenkins. Maven y cherche d'abord avant de télécharger depuis Internet.

Maven Central

Dépôt public hébergeant 500 000+ artefacts. Accessible à l'adresse https://search.maven.org. Référence mondiale pour les bibliothèques Java open-source.

Résolution des dépendances

Maven cherche dans : 1. Dépôt local, 2. Dépôts configurés dans pom.xml, 3. Maven Central. Télécharge et met en cache si absent.

En entreprise

En contexte professionnel, on utilise un proxy (Nexus, Artifactory) pour contrôler les dépendances autorisées et éviter les téléchargements directs depuis Internet.

Les gestionnaires d'artefacts : Nexus et Artifactory

Rôle

Un gestionnaire d'artefacts est un serveur qui stocke et distribue les artefacts logiciels (JARs, WARs, images Docker, packages npm...) produits par les builds.

Proxy du dépôt central

Intercepte les téléchargements Maven Central, les met en cache localement. Avantages : builds plus rapides, indépendance réseau, contrôle des versions autorisées.

Stockage des artefacts internes

Héberge les JARs produits par vos propres projets, versionnés et accessibles par d'autres projets de l'organisation.

Nexus vs Artifactory

Nexus Repository OSS (gratuit, JFrog Nexus) et JFrog Artifactory (commercial, plus riche) sont les deux solutions leaders du marché.

Intégration Maven dans Jenkins

Exemple de Jenkinsfile utilisant Maven via un agent Docker pour compiler et tester un projet Java.

```
pipeline {
    agent {
        docker {
            // Utiliser l'image Maven officielle comme agent
            image 'maven:3.9-eclipse-temurin-17'
            // Monter le cache Maven local pour accélérer les builds
            args '-v /root/.m2:/root/.m2'
        }
    }

    stages {
        stage('Compilation') {
            steps {
                sh 'mvn clean compile -B'
            }
        }
        stage('Tests') {
            steps {
                sh 'mvn test -B'
            }
            post {
                always {
                    // Publier les résultats des tests dans Jenkins
                    junit 'target/surefire-reports/*.xml'
                }
            }
        }
        stage('Packaging') {
            steps {
                sh 'mvn package -DskipTests -B'
                archiveArtifacts artifacts: 'target/*.jar'
            }
        }
    }
}
```

TP 2

TP 2 — Compilation d'un projet Java avec Maven

Créer un job Pipeline Jenkins pour cloner un projet Java, le compiler avec Maven et exécuter les tests unitaires JUnit.

💡 **Indice :** *Utilisez l'agent Docker avec l'image maven:3.9-eclipse-temurin-17 et publiez les résultats JUnit.*

À vous de jouer !

05
La conteneurisation

Docker dans le pipeline CI/CD

Virtualisation vs Conteneurisation

Machine Virtuelle (VM)

- Hyperviseur (VMware, VirtualBox, KVM)
- Chaque VM a son propre OS complet
- Démarrage en minutes
- Isolation forte (noyau séparé)
- Consommation : Go de RAM, dizaines de Go de disque

Conteneur Docker

- Partage le noyau Linux de l'hôte
- Isolation via namespaces et cgroups
- Démarrage en secondes
- Léger (Mo de RAM, Mo de disque)
- Portable : même comportement partout

Qu'est-ce que Docker ?

Concept	Description
Image Docker	Template immuable et en couches (layers) décrivant un environnement d'exécution. Définie par un Dockerfile.
Conteneur	Instance en cours d'exécution d'une image. Isolé du système hôte. Éphémère par nature.
Dockerfile	Fichier texte contenant les instructions pour construire une image (FROM, RUN, COPY, CMD...).
Registry (Registre)	Serveur de stockage et distribution d'images. Docker Hub (public), Harbor, GitLab Registry (privés).
Docker Engine	Daemon qui gère la création, l'exécution et la suppression des conteneurs sur la machine hôte.
Docker Compose	Outil pour définir et lancer des applications multi-conteneurs via un fichier YAML.

Les commandes Docker essentielles

Commande	Description
docker pull <image>	Télécharger une image depuis un registre
docker run -d -p 8080:8080 <image>	Démarrer un conteneur en arrière-plan
docker build -t <nom>:<tag> .	Construire une image depuis le Dockerfile courant
docker images	Lister les images disponibles localement
docker ps	Lister les conteneurs en cours d'exécution
docker ps -a	Lister tous les conteneurs (y compris arrêtés)
docker stop <id>	Arrêter un conteneur
docker rm <id>	Supprimer un conteneur arrêté
docker logs <id>	Afficher les logs d'un conteneur
docker exec -it <id> bash	Ouvrir un shell interactif dans un conteneur

Le Dockerfile : construire une image

Exemple de Dockerfile pour une application Java. Chaque instruction crée une nouvelle couche (layer) dans l'image.

```
# Image de base : JDK 17 sur Alpine Linux (léger)
FROM eclipse-temurin:17-jdk-alpine

# Répertoire de travail dans le conteneur
WORKDIR /app

# Copier le JAR compilé dans le conteneur
COPY target/demo-java-app-1.0.0.jar app.jar

# Exposer le port 8080 (documentation)
EXPOSE 8080

# Commande exécutée au démarrage du conteneur
CMD ["java", "-jar", "app.jar"]

# Construction : docker build -t demo-java-app:1.0 .
# Exécution  : docker run -d -p 8080:8080 demo-java-app:1.0
```

Le build multi-étapes (multi-stage)

Le multi-stage build permet de compiler avec un JDK complet et de n'inclure que le JRE dans l'image finale — image plus légère et plus sécurisée.

```
# ---- ÉTAPE 1 : Compilation ----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
# Copier d'abord le pom.xml (cache des dépendances Maven)
COPY pom.xml .
RUN mvn dependency:go-offline -B
# Copier le code source et compiler
COPY src ./src
RUN mvn clean package -DskipTests -B

# ---- ÉTAPE 2 : Image finale (légère) ----
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
# Copier UNIQUEMENT le JAR depuis l'étape de build
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
# Résultat : image ~200MB au lieu de ~600MB avec le JDK complet
```

Docker Hub et les registres d'images

Docker Hub

Registre public officiel de Docker (hub.docker.com). Héberge des images officielles (ubuntu, nginx, mysql, maven, jenkins...) et des images communautaires.

Registres privés

En entreprise, on utilise un registre privé pour héberger les images internes : Harbor (open-source), JFrog Artifactory, GitLab Container Registry, AWS ECR.

Commandes

docker push <registre>/<image>:<tag> pour publier, docker pull <registre>/<image>:<tag> pour télécharger.

Bonnes pratiques

Toujours tagger les images avec une version (pas seulement :latest), utiliser des images officielles comme base, scanner les images pour les vulnérabilités.

Docker Compose : orchestrer plusieurs conteneurs

Exemple d'un docker-compose.yml pour lancer Jenkins et SonarQube ensemble pour les TPs.

```yaml
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts-jdk17
    container_name: jenkins
    network_mode: host
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    restart: unless-stopped

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    network_mode: host
    environment:
      - SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true
    volumes:
      - sonar_data:/opt/sonarqube/data
    restart: unless-stopped

volumes:
  jenkins_home:
  sonar_data:

# Démarrer : docker compose up -d
# Arrêter  : docker compose down
```

Utiliser Docker dans un pipeline Jenkins

Deux usages de Docker dans Jenkins : comme agent d'exécution, et pour construire des images applicatives.

```
pipeline {
    // Utiliser Docker comme agent d'exécution
    agent {
        docker {
            image 'maven:3.9-eclipse-temurin-17'
            args '-v /root/.m2:/root/.m2'
        }
    }

    stages {
        stage('Compilation & Tests') {
            steps {
                sh 'mvn clean package -B'
            }
        }

        // Construire une image Docker de l'application
        stage('Build Image Docker') {
            agent { label 'docker-agent' }
            steps {
                script {
                    def image = docker.build("demo-java-app:${BUILD_NUMBER}")
                    // Pousser vers un registre
                    docker.withRegistry('https://registry.example.com', 'registry-creds') {
                        image.push()
                        image.push('latest')
                    }
                }
            }
        }
    }
}
```

TP 3

TP 3 — Conteneurisation de l'application Java

Écrire un Dockerfile multi-stage pour l'application Java et intégrer une étape de construction d'image Docker dans le pipeline Jenkins.

💡 **Indice :** *Montez le socket Docker (/var/run/docker.sock) dans le conteneur Jenkins pour lui donner accès au daemon Docker de l'hôte.*

À vous de jouer !

06
Sécurité et qualité du code

SonarQube, JUnit, JaCoCo

Pourquoi la qualité du code est-elle critique ?

La règle du coût exponentiel

Corriger un bug coûte 1x en développement, 10x en test, 100x en production. Détecter les problèmes tôt est économiquement vital.

La dette technique

Code mal structuré, dupliqué ou non testé qui s'accumule et ralentit les développements futurs. Estimée à 40% du temps de développement perdu.

Enjeux de sécurité

Les failles de sécurité (injections SQL, XSS, secrets en dur...) peuvent être détectées par analyse statique avant qu'elles n'atteignent la production.

OWASP Top 10

Liste des 10 risques de sécurité les plus critiques pour les applications web. SonarQube intègre des règles basées sur l'OWASP.

Les métriques de qualité logicielle

Métrique	Définition	Outil
Bugs	Erreurs fonctionnelles certaines dans le code	SonarQube
Vulnerabilities	Failles de sécurité exploitables	SonarQube
Security Hotspots	Code sensible nécessitant une revue manuelle	SonarQube
Code Smells	Mauvaises pratiques réduisant la maintenabilité	SonarQube
Coverage	% du code couvert par des tests automatisés	JaCoCo + SonarQube
Duplications	% de code dupliqué (copier-coller)	SonarQube
Complexity	Complexité cyclomatique des méthodes	SonarQube

Introduction à SonarQube

Définition

SonarQube est une plateforme open-source d'inspection continue de la qualité du code. Elle analyse statiquement le code source et produit un rapport détaillé.

Fonctionnement

Un scanner (SonarScanner ou plugin Maven) analyse le code et envoie les résultats au serveur SonarQube. Le tableau de bord affiche les métriques et l'historique.

Langages supportés

Java, JavaScript, TypeScript, Python, C#, C/C++, PHP, Go, Kotlin, Scala... Plus de 30 langages supportés.

Éditions

Community Edition (gratuite, open-source), Developer Edition (branches, sécurité avancée), Enterprise Edition (portfolios, gouvernance).

Le tableau de bord SonarQube

Vue projet

Résumé des métriques clés : Reliability (Bugs), Security (Vulnerabilities), Maintainability (Code Smells), Coverage, Duplications.

New Code vs Overall Code

SonarQube distingue le nouveau code (depuis la dernière version) du code global. Le Quality Gate s'applique généralement au nouveau code.

Détail des issues

Chaque problème est listé avec sa sévérité (Blocker, Critical, Major, Minor, Info), sa localisation précise et une explication pédagogique.

Historique

Graphiques d'évolution des métriques dans le temps. Permet de mesurer l'amélioration ou la dégradation de la qualité du code.

Le Quality Gate : la porte de qualité

Définition

Un Quality Gate est un ensemble de conditions (seuils) que le code doit respecter pour être considéré comme 'passant'. Ex: Coverage >= 80%, 0 Bugs critiques.

Intégration CI/CD

Dans le pipeline Jenkins, l'étape waitForQualityGate() met en pause le pipeline et attend le résultat du Quality Gate. Si KO, le build échoue.

Sonar Way

Quality Gate par défaut de SonarQube. Conditions : 0 nouveaux Bugs, 0 nouvelles Vulnerabilities, Coverage >= 80% sur le nouveau code.

Configuration

Les Quality Gates sont configurables dans SonarQube (Administration > Quality Gates). On peut créer des gates personnalisées par projet ou par équipe.

Les tests unitaires avec JUnit et JaCoCo

JUnit 5

Framework de test unitaire standard en Java. Annotations clés : @Test, @BeforeEach, @AfterEach, @ParameterizedTest. Assertions : assertEquals(), assertTrue(), assertThrows().

Mockito

Bibliothèque de mocking pour isoler les classes testées de leurs dépendances. Permet de simuler le comportement d'objets externes (bases de données, APIs...).

JaCoCo

Java Code Coverage — mesure le pourcentage de code couvert par les tests. Génère des rapports HTML et XML. Intégré à Maven via le plugin jacoco-maven-plugin.

Dans Jenkins

Les résultats JUnit sont publiés avec junit 'target/surefire-reports/*.xml'. Les rapports JaCoCo sont envoyés à SonarQube lors de l'analyse.

Intégration SonarQube dans Jenkins

Étapes pour intégrer SonarQube dans un pipeline Jenkins avec Quality Gate.

```
// 1. Prérequis : Plugin "SonarQube Scanner" installé dans Jenkins
// 2. Configurer le serveur SonarQube dans Manage Jenkins > System
// 3. Ajouter le token SonarQube dans Manage Jenkins > Credentials

pipeline {
    agent {
        docker {
            image 'maven:3.9-eclipse-temurin-17'
            args '--network host'
        }
    }
    stages {
        stage('Analyse SonarQube') {
            steps {
                // withSonarQubeEnv injecte les variables SONAR_HOST_URL et SONAR_TOKEN
                withSonarQubeEnv('SonarQube') {
                    sh 'mvn sonar:sonar -Dsonar.projectKey=demo-java-app'
                }
            }
        }
        stage('Quality Gate') {
            steps {
                // Attendre le résultat du Quality Gate (webhook SonarQube requis)
                timeout(time: 5, unit: 'MINUTES') {
                    waitForQualityGate abortPipeline: true
                }
            }
        }
    }
}
```

La sécurité dans Jenkins

Gestion des credentials

Ne JAMAIS mettre de mots de passe ou tokens en clair dans un Jenkinsfile. Utiliser le Credentials Store de Jenkins et withCredentials() ou les variables d'environnement.

Contrôle d'accès

Configurer la Matrix Authorization Strategy pour définir des rôles (admin, développeur, lecteur) et restreindre les actions par utilisateur ou groupe.

Sécurisation des agents

Utiliser des agents éphémères (Docker, Kubernetes) pour isoler les builds. Ne jamais exécuter les builds directement sur le Controller.

Audit des plugins

Maintenir les plugins à jour, supprimer les plugins inutilisés, vérifier les CVE (Common Vulnerabilities and Exposures) régulièrement.

TP 4

TP 4 — Analyse de qualité avec SonarQube

Démarrer SonarQube via Docker, générer un token d'authentification, configurer l'intégration dans Jenkins et ajouter l'analyse de code dans le pipeline.

💡 **Indice :** *Démarrez SonarQube avec docker run -d --name sonarqube --network host sonarqube:lts-community et attendez qu'il soit prêt sur le port 9000.*

À vous de jouer !

07
Simplifier l'intégration

Outils avancés et bonnes pratiques

L'écosystème CI/CD complet

Le flux complet

GitLab (SCM + MR) → Webhook → Jenkins (Orchestration) → Maven (Build + Test) → SonarQube (Qualité) → Docker (Packaging) → Nexus (Artefacts) → Déploiement.

Chaque outil à sa place

GitLab gère le code, Jenkins orchestre, Maven compile, SonarQube contrôle la qualité, Docker packagise, Nexus stocke. Chaque outil est le meilleur dans son domaine.

Infrastructure as Code

Toute la configuration (Jenkinsfile, Dockerfile, docker-compose.yml, .gitlab-ci.yml) est versionnée dans Git. L'infrastructure est reproductible et auditée.

Observabilité

Les métriques de build (durée, taux d'échec, couverture de tests) sont des indicateurs de santé de l'équipe. Surveillez-les dans le temps.

Les Webhooks : automatiser les déclenchements

Principe

Un Webhook est une notification HTTP envoyée par GitLab à Jenkins à chaque événement (push, MR, tag). Jenkins déclenche immédiatement le pipeline correspondant.

Configuration GitLab

Settings > Webhooks > Add new webhook. URL : http://<jenkins-url>/generic-webhook-trigger/invoke ou /project/<job-name>. Événements : Push events, Merge request events.

Configuration Jenkins

Activer 'Build when a change is pushed to GitLab' dans la configuration du job. Ou utiliser le plugin 'Generic Webhook Trigger' pour plus de flexibilité.

Avantage vs Polling

Le Webhook est instantané et ne génère pas de charge sur Jenkins. Le Polling interroge régulièrement et peut manquer des événements ou créer des délais.

Les pipelines multi-branches

Définition

Un Multibranch Pipeline Jenkins scanne automatiquement toutes les branches d'un dépôt Git et crée un job Jenkins pour chaque branche contenant un Jenkinsfile.

Avantages

Pas de configuration manuelle pour les nouvelles branches, chaque branche a son propre historique de builds, les branches supprimées voient leur job supprimé automatiquement.

Comportement par branche

On peut adapter le pipeline selon la branche : déployer en production uniquement depuis main, en staging depuis develop, juste compiler/tester pour les features.

Configuration

Dans Jenkins : New Item > Multibranch Pipeline. Configurer la source Git (URL du dépôt, credentials). Jenkins scanne et crée les jobs automatiquement.

Les variables d'environnement et credentials

Bonnes pratiques pour gérer les secrets et variables dans Jenkins — ne jamais mettre de secrets en clair dans le Jenkinsfile.

```
pipeline {
    agent any

    environment {
        // Variables d'environnement disponibles dans tout le pipeline
        APP_NAME    = 'demo-java-app'
        REGISTRY    = 'registry.example.com'
        // Référencer un credential Jenkins (jamais en clair !)
        SONAR_TOKEN = credentials('sonar-token')
    }

    stages {
        stage('Deploy') {
            steps {
                // Utiliser withCredentials pour injecter des secrets
                withCredentials([
                    usernamePassword(
                        credentialsId: 'registry-credentials',
                        usernameVariable: 'REGISTRY_USER',
                        passwordVariable: 'REGISTRY_PASS'
                    )
                ]) {
                    sh '''
                        echo $REGISTRY_PASS | docker login $REGISTRY -u $REGISTRY_USER --password-stdin
                        docker push $REGISTRY/$APP_NAME:latest
                    '''
                }
            }
        }
    }
}
```

Les notifications : rester informé

Email (Email Extension Plugin)

Envoyer des emails de notification en cas d'échec, de retour à la normale ou à chaque build. Configurable dans le bloc post{} du Jenkinsfile.

Microsoft Teams / Slack

Plugins Office 365 Connector (Teams) et Slack Notification Plugin. Envoient des messages dans un canal avec le statut du build et un lien vers les logs.

Dans le Jenkinsfile

post { failure { emailext to: 'equipe@sncf.fr', subject: 'Build échoué', body: '...' } success { slackSend channel: '#ci', message: 'Build OK' } }

Bonne pratique

Notifier uniquement en cas de changement d'état (succès → échec ou échec → succès) pour éviter le bruit. Utiliser la condition 'changed' dans le bloc post{}.

Les pipelines parallèles

L'instruction parallel{} permet d'exécuter plusieurs stages simultanément, réduisant significativement la durée totale du pipeline.

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps { sh 'mvn clean package -DskipTests -B' }
        }

        // Exécution parallèle : Tests + Analyse en même temps
        stage('Vérifications parallèles') {
            parallel {
                stage('Tests Unitaires') {
                    agent { docker { image 'maven:3.9-eclipse-temurin-17' } }
                    steps {
                        sh 'mvn test -B'
                        junit 'target/surefire-reports/*.xml'
                    }
                }
                stage('Analyse SonarQube') {
                    agent { docker { image 'maven:3.9-eclipse-temurin-17' } }
                    steps {
                        withSonarQubeEnv('SonarQube') {
                            sh 'mvn sonar:sonar'
                        }
                    }
                }
            }
        }

        stage('Deploy') {
            steps { echo 'Déploiement...' }
        }
    }
}
```

Blue Ocean : une interface moderne pour Jenkins

Présentation

Blue Ocean est un plugin Jenkins offrant une interface graphique moderne et intuitive, conçue spécifiquement pour les pipelines. Accessible via /blue sur l'URL Jenkins.

Visualisation des pipelines

Représentation graphique du pipeline sous forme de graphe orienté. Chaque stage est une boîte colorée (vert=succès, rouge=échec). Navigation facile dans les logs.

Éditeur visuel

Blue Ocean inclut un éditeur visuel permettant de créer et modifier des Jenkinsfiles sans écrire de code Groovy, directement depuis l'interface web.

Vue par branche

Dans un Multibranch Pipeline, Blue Ocean affiche l'état de toutes les branches et Pull Requests en un coup d'œil.

Jenkins et Kubernetes : aller plus loin

Problème des agents fixes

Les agents Jenkins permanents sont souvent sous-utilisés ou saturés. Ils nécessitent une maintenance et ne s'adaptent pas à la charge.

Solution : agents Kubernetes

Le plugin Kubernetes pour Jenkins crée des agents éphémères (pods) à la demande dans un cluster Kubernetes. Chaque build a son propre pod, supprimé après le build.

Avantages

Scalabilité automatique (autant d'agents que nécessaire), isolation parfaite (pod dédié par build), pas de maintenance des agents, utilisation optimale des ressources.

Déclaration dans le Jenkinsfile

agent { kubernetes { yaml ''' spec: containers: - name: maven image: maven:3.9-eclipse-temurin-17 ''' } }

TP 5

TP 5 — Le pipeline CI/CD complet

Assembler toutes les étapes (Checkout, Compilation, Tests, SonarQube, Quality Gate, Packaging, Build Docker) dans un pipeline CI/CD complet et robuste.

💡 **Indice :** *Ajoutez un bloc post{} avec cleanWs() et configurez le Webhook SonarQube (http://localhost:8080/sonarqube-webhook/) pour que le Quality Gate fonctionne.*

À vous de jouer !

Récapitulatif du cours

Chapitre	Outils	Compétences acquises
1. CI/CD	—	Comprendre DevOps, CI, CD, pipeline
2. GitLab	Git, GitLab	Versionner, brancher, faire des MR
3. Jenkins	Jenkins, Groovy	Créer des pipelines, Jenkinsfile
4. Dépendances	Maven, Nexus	Gérer les builds Java, pom.xml
5. Docker	Docker, Compose	Conteneuriser, multi-stage, Dockerfile
6. Qualité	SonarQube, JUnit	Analyser le code, Quality Gate
7. Outils avancés	Webhooks, Blue Ocean	Automatiser, paralléliser, notifier

Les 8 bonnes pratiques CI/CD

1. Commits fréquents	Intégrez votre code au moins une fois par jour. Des commits petits et fréquents sont plus faciles à revoir et à déboguer.
2. Builds rapides	Un pipeline CI doit s'exécuter en moins de 10 minutes. Parallélisez les stages, utilisez le cache Maven/npm, optimisez les images Docker.
3. Pipeline as Code	Versionnez votre Jenkinsfile avec le code source. Traitez votre pipeline comme du code : revue, tests, historique.
4. Jamais de secrets en clair	Utilisez le Credentials Store de Jenkins. Auditez régulièrement les secrets et révoquez ceux qui ne sont plus utilisés.
5. Quality Gate obligatoire	Un build qui ne passe pas le Quality Gate ne doit pas être déployé. La qualité est non-négociable.
6. Tests en premier	Écrivez les tests avant ou en même temps que le code (TDD). Un code sans tests n'est pas du code terminé.
7. Notifications intelligentes	Notifiez uniquement lors des changements d'état. Trop de notifications = plus personne ne les lit.
8. Monitorer les métriques	Suivez le taux d'échec des builds, la durée des pipelines, la couverture de tests. Ce sont des indicateurs de santé de l'équipe.

Merci !

Des questions ?

Ressources pour aller plus loin :

Documentation Jenkins : https://www.jenkins.io/doc/

Documentation GitLab CI/CD : https://docs.gitlab.com/ee/ci/

Documentation Docker : https://docs.docker.com/

Documentation SonarQube : https://docs.sonarqube.org/

Documentation Maven : https://maven.apache.org/guides/